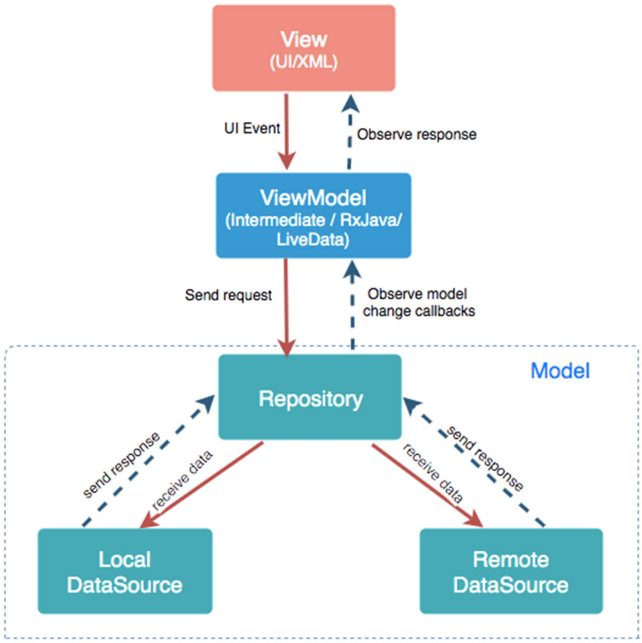
MVVM Architecture

MVVM architecture is a Model-View-ViewModel architecture, where **View** represents an activity or fragment, **Model** represents a component responsible for bringing data into the application from API's and database, and **ViewModel** is the layer in between of them.



Lifecycle-aware components : Lifecycle-aware components is a component which perform actions in response to a change in the lifecycle status of ~~another component, such as~~ activities and fragments. Such a component is also called as LifecycleObserver.

```
public class MyObserver implements LifecycleObserver {

    @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)
    public void connectListener() {
        ...
    }

    @OnLifecycleEvent(Lifecycle.Event.ON_PAUSE)
    public void disconnectListener() {
        ...
    }
}
```

A lifecycle observer can be created by implementing LifeCycleObserver and by adding annotations to its methods. Then you can add this observer to the activity class by using `addObserver()` method

Inside activity :

```
myLifecycleOwner.getLifecycle().addObserver(new MyObserver());
```

ViewModel

The `ViewModel` is a component which lives outside the lifecycle of an activity or a fragment, thus allows data to survive configuration changes such as screen rotations. A View model can be created by extending the **ViewModel** class.

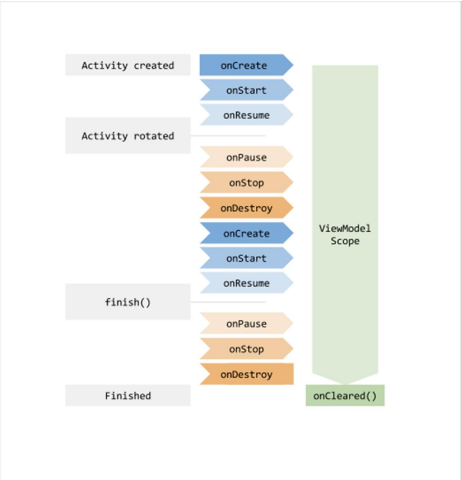
```
public class MainActivityViewModel extends ViewModel implements LifecycleObserver {

    MutableLiveData<Integer> counter;
    public MainActivityViewModel(){

    }


    public LiveData<Integer> getCounter(){
        return counter;
    }
}
```

ViewModel objects are automatically retained during configuration changes so that data they hold is immediately available to the next activity or fragment instance.



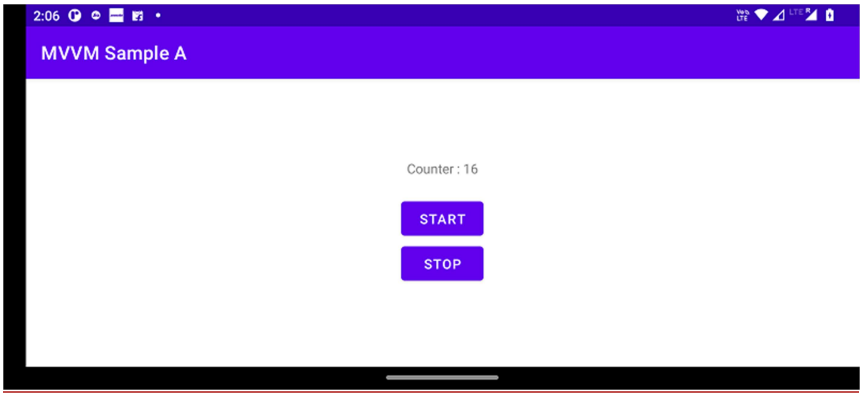
Once created, we can add an observer over a function of a View Model to get the data

```
viewModel = new ViewModelProvider(this).get(MainActivityViewModel.class);

viewModel.getCounter().observe(this, new Observer<Integer>() {
    @Override
    public void onChanged(Integer integer) {
        textViewCounter.setText("Counter : " + integer);
    }
});
```

LiveData

LiveData is a data holder class that can be observed within a given lifecycle. This means that an `Observer` can be added in a pair with a `LifecycleOwner`, and this observer will be notified about modifications of the wrapped data only if the paired LifecycleOwner is in active state.



MainActivity.java

```
package com.example.mvvmsamplea;

import androidx.appcompat.app.AppCompatActivity;
import androidx.lifecycle.Observer;
import androidx.lifecycle.ViewModel;
import androidx.lifecycle.ViewModelProvider;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity {

    private static final String TAG = MainActivity.class.getSimpleName();
    Button startButton, stopButton;
    TextView textViewCounter;

    MainActivityViewModel viewModel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        viewModel = new ViewModelProvider(this).get(MainActivityViewModel.class);

        viewModel.getCounter().observe(this, new Observer<Integer>() {
            @Override
            public void onChanged(Integer integer) {
                textViewCounter.setText("Counter : " + integer);
            }
        });

        //--life cycle observer
        getLifecycle().addObserver(viewModel);

        textViewCounter = (TextView) findViewById(R.id.textViewCounter);
        startButton = (Button) findViewById(R.id.startBtn);
        stopButton = (Button) findViewById(R.id.stopBtn);

        startButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                viewModel.startCounter();
            }
        });

        stopButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                viewModel.stopCounter();
            }
        });
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
    }
}
```

MainActivityViewModel.java

```java
package com.example.mvvmsamplea;

import androidx.annotation.NonNull;
import androidx.lifecycle.Lifecycle;
import androidx.lifecycle.LifecycleEventObserver;
import androidx.lifecycle.LifecycleObserver;
import androidx.lifecycle.LifecycleOwner;
import androidx.lifecycle.LiveData;
import androidx.lifecycle.MutableLiveData;
import androidx.lifecycle.OnLifecycleEvent;
import androidx.lifecycle.ViewModel;

public class MainActivityViewModel extends ViewModel implements LifecycleObserver {

    Thread counterThread;
    int count = 0;
    boolean isCounterInProgress;

    MutableLiveData<Integer> counter;

    public MainActivityViewModel(){
        isCounterInProgress = false;

        counter = new MutableLiveData<>();

        counterThread = new Thread((Runnable) () -> {
            while (isCounterInProgress){
                try{
                    Thread.sleep(1000);
                    count++;
                    counter.postValue(count);
                }catch (InterruptedException e){
                    e.printStackTrace();
                }
            }
        });
    }

    public LiveData<Integer> getCounter(){
        return counter;
    }

    public void startCounter(){
        if(!isCounterInProgress){
            isCounterInProgress = true;
            counterThread.start();
        }
    }

    public void stopCounter(){
        isCounterInProgress = false;
    }

    //-- life cycle observer method
    /*@Override
    public void onStateChanged(@NonNull LifecycleOwner source, @NonNull Lifecycle.Event event) {
        System.out.println("onStateChanged is executed.");
        switch (event){
            case ON_CREATE:
                break;
            case ON_START:
                break;
            case ON_RESUME:
                System.out.println("Resume method executed.");
                break;
            case ON_PAUSE:
                break;
            case ON_STOP:
                break;
            case ON_DESTROY:
                break;
            case ON_ANY:
                throw new IllegalArgumentException("ON_ANY must not been send by anybody");
        }
    }*/

    @OnLifecycleEvent(Lifecycle.Event.ON_RESUME)
    public void onRefresh(){
        System.out.println("Resume method executed.");
    }
}
```